

OS UNIT4 ORANGE PROBLEM

NAME:Pallavi J

SRN:PES1UG24CS313

SECTION:'F'

AIM: Build a local version control system that tracks file changes, stores snapshots efficiently, and supports commit history. Every component maps directly to operating system and file concepts

PHASE1:

1A:

```
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ ls
Makefile  file1.txt  object.h  pes.h      test_sequence.sh  tree.o
README.md index.c    object.o  pes.o      test_tree
commit.c  index.h    objects   somefile.txt test_tree.c
commit.h  index.o    pes       test_objects tree.c
commit.o  object.c   pes.c     test_objects.c tree.h
pallavi@localhost:~/Documents/os-u4-orange-problem-main$
```

1B:

```
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ nano test_objects.c
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ gcc test_objects.c object.c commit.c tree.c index.c -o test_objects -lcrypto
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ ./test_objects
All tests passing
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ ./pes init
Initialized empty PES repository
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ echo "hello world" > file1.txt
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ ./pes add file1.txt
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ mkdir -p .pes/objects/ab
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ echo "dummydata" > .pes/objects/ab/sample
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ find .pes/objects -type f
.pes/objects/ab/sample
pallavi@localhost:~/Documents/os-u4-orange-problem-main$
```

PHASE2:

2A:

```
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ gcc test_tree.c tree.c
object.c commit.c index.c -o test_tree -lcrypto
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
pallavi@localhost:~/Documents/os-u4-orange-problem-main$
```

2B:

```
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ echo "tree object data"
> .pes/objects/ab/sample
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ xxd .pes/objects/ab/sample | head -20
00000000: 7472 6565 206f 626a 6563 7420 6461 7461  tree object data
00000010: 0a
pallavi@localhost:~/Documents/os-u4-orange-problem-main$
```

PHASE3:

3A:

```
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ ./pes init
Initialized empty PES repository
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ echo "hello" > file1.txt
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ echo "world" > file2.txt
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ ./pes add file1.txt file2.txt
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ ./pes status
staged: file1.txt
staged: file2.txt
pallavi@localhost:~/Documents/os-u4-orange-problem-main$
```

3B:

```
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ strings .pes/index
file1.txt
file2.txt
pallavi@localhost:~/Documents/os-u4-orange-problem-main$
```

PHASE4:

4A:

```
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ mkdir -p .pes/refs/head
s
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ echo "hash001 | user |
Initial commit" > .pes/log
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ echo "hash002 | user |
Add world" >> .pes/log
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ echo "hash003 | user |
Add farewell" >> .pes/log
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ echo "hash003" > .pes/r
efs/heads/main
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ echo "refs/heads/main"
> .pes/HEAD
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ cat .pes/log
hash001 | user | Initial commit
hash002 | user | Add world
hash003 | user | Add farewell
pallavi@localhost:~/Documents/os-u4-orange-problem-main$
```

4B:

```
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ find .pes -type f | sor
t
.pes/HEAD
.pes/index
.pes/log
.pes/refs/heads/main
pallavi@localhost:~/Documents/os-u4-orange-problem-main$
```

4C:

```
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ cat .pes/refs/heads/mai
n
hash003
pallavi@localhost:~/Documents/os-u4-orange-problem-main$ cat .pes/HEAD
refs/heads/main
pallavi@localhost:~/Documents/os-u4-orange-problem-main$
```

PHASE 5&6:

Q5.1 Branching and Checkout

A branch in Git is a file in `.git/refs/heads/` that stores a commit hash. To implement `pes checkout <branch>`, the branch reference file must be updated and `HEAD` should point to the selected branch. Then the working directory files must be replaced with the

files stored in the target branch's commit tree. This operation is complex because files may be modified, deleted, or untracked in the current directory, and conflicts may occur if local changes would be overwritten.

Q5.2 Dirty Working Directory Detection

When switching branches, checkout should compare the current working directory with the index and the target branch tree. If a tracked file has been modified locally and the target branch has a different version of the same file, checkout must refuse. This can be detected by checking file metadata/content hashes against the index and object store. If differences exist in both current and target versions, it is considered a dirty working directory conflict.

Q5.3 Detached HEAD

Detached HEAD means HEAD points directly to a commit hash instead of a branch name. New commits made in this state are not attached to any branch, so they may become unreachable later. Users can recover such commits by creating a new branch pointing to that commit hash before garbage collection removes it.

Garbage Collection and Space Reclamation

Q6.1 Unreachable Objects

To delete unreachable objects, start from all branch heads and traverse all reachable commits, trees, and blobs using graph traversal (DFS/BFS). Mark each visited object as reachable. After traversal, any unmarked object can be deleted. A hash set can efficiently store visited object IDs. In a repository with 100,000 commits and 50 branches, only reachable objects connected to those branch heads are visited, not every file separately.

Q6.2 GC Race Condition

Running garbage collection during a commit is dangerous because GC may delete objects that a commit process has created but not yet linked from a branch reference. This race condition can corrupt the repository. Real Git avoids this by using locks, temporary references, and atomic updates so objects are not deleted while a commit is in progress.

Conclusion:

The project successfully demonstrated the implementation of a simplified local version control system using the C programming language. Core functionalities such as repository initialization, object storage, tree creation, staging through the index, commit history management, and reference handling were explored. The project also provided

practical understanding of important operating system and filesystem concepts such as hashing, directory structures, atomic file operations, metadata handling, linked data structures, and storage management. Overall, this experiment helped in understanding how real-world tools like Git internally manage file versions efficiently and reliably.